

Week4

代码质量；元编程；大杂烩；课堂综合测试

姓名：赵均临 学号：25020007178

日期：2026 年 9 月 16 日

目录

1	课上实验题目	3
1.1	实验13：建立可执行的本地质量门禁	3
1.1.1	实验目的	3
1.1.2	实验环境	3
1.1.3	实验步骤	3
1.1.4	实验结果	5
1.2	实验14：让MAKE只重建真正受影响的产物	6
1.2.1	实验目的	6
1.2.2	依赖关系梳理	6
1.2.3	实验实现	6
1.3	实验15：把本地API数据转换为可读报告	7
1.3.1	实验目的	7
1.3.2	核心逻辑设计	7
1.3.3	核心代码片段	7
1.3.4	完整生成脚本	7
1.3.5	验证结果	8
1.3.6	扩展说明	8
1.4	实验16：修复并交付一个小型的陌生仓库	8
1.4.1	实验目的	8
1.4.2	操作流程	8
1.4.3	Git提交记录	10
1.4.4	踩坑总结	10
2	练习	11
2.1	代码质量与自动化测试	11
2.1.1	配置格式化器、linter 和 pre-commit 钩子	11
2.1.2	编写单元测试并生成覆盖率报告	11
2.1.3	设置持续集成（CI）	12

2.1.4	使用 grep 和 semgrep 查找危险代码	12
2.2	正则表达式与元编程	12
2.2.1	IDE 正则表达式替换 Markdown 符号	12
2.2.2	正则提取 JSON 中的 name 并修复贪婪匹配	12
2.2.3	使用 JSON 解析器提取 name	13
2.3	构建系统与 Git 钩子	13
2.3.1	Makefile 实现 clean 伪目标	13
2.3.2	Git pre-commit 钩子拒绝构建失败	13
2.3.3	GitHub Action 检查 shell 和 markdown	13

github网址: <https://github.com/coolicer17/Systemtools>

1 课上实验题目

1.1 实验13: 建立可执行的本地质量门禁

1.1.1 实验目的

掌握Python项目轻量质量门禁搭建方法, 通过ruff实现代码格式/规范检查, 结合pytest单元测试, 最终实现一键校验脚本, 保证代码合入前符合质量要求。

1.1.2 实验环境

- Python 3.10+ - 依赖包: ruff>=0.4.0, pytest>=8.0.0 - 基础项目: 复用实验10完成的greetlab问候包

1.1.3 实验步骤

3.1 项目迁移

复制实验10的项目为实验13工作目录:

```
1 cp -r ./q10 ./q13 && cd q13
```

3.2 配置质量工具

在pyproject.toml中添加最小配置, 无需额外配置文件, 统一在项目元数据中管理规则:

```
1 [tool.ruff]
2 line-length = 119
3 target-version = "py310"
4 [tool.ruff.lint]
5 select = ["E", "F", "W", "I"]
6 [tool.pytest.ini_options]
7 testpaths = ["tests"]
```

```

20 [tool.ruff.lint]
21 # 启用基础代码规则：语法错误/格式规范/导入排序
22 select = ["E", "F", "W", "I"]
23 # 不得全局忽略规则，因此不配置ignore字段
24
25 [tool.ruff.format]
26 quote-style = "double"
27
28 [tool.pytest.ini_options]
29 testpaths = ["tests"] # 指定测试目录
30 addopts = "-v --color=yes" # 默认输出详细日志

```

3.3 补充单元测试用例

新增正常输入+边界（空白）输入测试，覆盖核心逻辑：

```

1 from greetlab import greet
2
3 def test_greet_normal_name():
4     assert greet("TestUser") == "Hello, TestUser!"
5
6 def test_greet_blank_name():
7     assert greet("   ") == "Hello, Guest!"
8     assert greet("") == "Hello, Guest!"

```

```

import pytest
from greetlab import greet

def test_greet_with_normal_name():
    """正常非空姓名返回匹配的问候语"""
    actual = greet("Alice")
    assert actual == "Hello, Alice!", f"正常姓名预期返回'Hello, Alice!', 实际得>

def test_greet_with_blank_name():
    """空字符串/全空格姓名返回默认访客问候"""
    # 覆盖空字符串场景
    assert greet("") == "Hello, Guest!", "空姓名需返回访客问候"
    # 覆盖全空白场景
    assert greet("   \t\n") == "Hello, Guest!", "纯空白姓名需返回访客问候"

```

3.4 修复质量问题

依次执行格式化、lint修复、测试运行，确保所有规则告警清零：

```

1 pip install ruff pytest
2 ruff format .
3 ruff check . --fix
4 pytest

```

```

===== test session starts =====
platform linux -- Python 3.10.12, pytest-9.1.1, pluggy-1.6.0 -- /usr/bin/python3
cachedir: .pytest_cache
rootdir: /home/coolicer/q13
configfile: pyproject.toml
testpaths: tests
plugins: anyio-4.12.1
collected 2 items

tests/test_greet.py::test_greet_normal_name PASSED [ 50%]
tests/test_greet.py::test_greet_blank_name PASSED [100%]

===== 2 passed in 0.01s =====
coolicer@1:~/q13$

```

3.5 编写门禁脚本check.sh

实现一键校验，任意环节失败直接终止并返回非0状态码：

```

1  #!/usr/bin/env bash
2  set -euo pipefail
3  echo "[Check_1/3]_Format_check"
4  ruff format --check .
5  echo "[Check_2/3]_Lint_check"
6  ruff check .
7  echo "[Check_3/3]_Unit_test"
8  pytest
9  echo "All_checks_passed!"

```

添加执行权限后验证返回值为0：

```

1  chmod +x check.sh
2  ./check.sh && echo "Return_code:_$?"

```

1.1.4 实验结果

```

coolicer@1:~/q13$ ./check.sh
===== 1/3 检查代码格式合规性 =====
5 files already formatted
===== 2/3 检查代码Lint规则 =====
All checks passed!
===== 3/3 运行单元测试 =====
===== test session starts =====
platform linux -- Python 3.10.12, pytest-9.1.1, pluggy-1.6.0 -- /usr/bin/python3
cachedir: .pytest_cache
rootdir: /home/coolicer/q13
configfile: pyproject.toml
testpaths: tests
plugins: anyio-4.12.1
collected 2 items

tests/test_greet.py::test_greet_normal_name PASSED [ 50%]
tests/test_greet.py::test_greet_blank_name PASSED [100%]

===== 2 passed in 0.01s =====
✅ 所有质量门禁全部通过！

```

1.2 实验14：让MAKE只重建真正受影响的产物

1.2.1 实验目的

理解Make构建系统的核心原理：通过对比目标与依赖的修改时间戳，实现**增量最小构建**，避免修改单文件后全量重复构建，大幅提升大型项目开发效率。

1.2.2 依赖关系梳理

本次项目的文件流向：

data.csv + stats.py → stats.txt

report.md + stats.txt + build_report.py → report.txt

其中‘data.csv/report.md/stats.py/build_report.py’是手写源文件（不会被构建生成），‘stats.txt/report.txt’是临时构建产物。

1.2.3 实验实现

3.1 Makefile实现

显式声明所有依赖，标记伪目标避免和实际文件冲突：

```
1 PYTHON ?= python3
2 PRODUCTS = stats.txt report.txt
3
4 .PHONY: all clean
5
6 all: $(PRODUCTS)
7
8 stats.txt: data.csv stats.py
9     $(PYTHON) stats.py
10
11 report.txt: report.md stats.txt build_report.py
12     $(PYTHON) build_report.py
13
14 clean:
15     rm -f $(PRODUCTS)
```

```
coolicer@1:~/q14$ make
python3 stats.py
python3 build_report.py
coolicer@1:~/q14$ make
make: Nothing to be done for 'all'.
```

3.2 验证结果

```
coolicer@1:~/q14$ touch data.csv
coolicer@1:~/q14$ make
python3 stats.py
python3 build_report.py
```

1.3 实验15：把本地API数据转换为可读报告

1.3.1 实验目的

掌握CLI工具链组合能力：通过本地HTTP服务提供JSON数据，用‘curl’拉取、‘jq’做数据过滤/排序/转换，最终生成符合Markdown规范的静态报告，实现无代码依赖的轻量数据处理。

1.3.2 核心逻辑设计

— 环节 — 实现工具 — 规则 —

— 本地服务提供JSON — Python内置‘http.server’ — 零依赖启动静态文件服务 —
 — 拉取数据 — ‘curl -fsS’ — 静默模式，请求失败直接报错退出 —
 — 数据处理 — ‘jq’ — 筛选‘status=active’且‘downloads≥100’
 — 报告输出 — Shell重定向 — 拼接表头+处理后的内容生成‘.md’文件 —

1.3.3 核心代码片段

jq处理逻辑

```
1 .[] | select(.status == "active" and .downloads >= 100)
2 | sort_by(-.downloads, .name)
3 | "|_\"(.name)\"_|_\"(.version)\"_|_\"(.downloads)\"_|"
```

1.3.4 完整生成脚本

```
1 set -euo pipefail
2 API="http://127.0.0.1:8000/packages.json"
3 { echo "#_Active_Packages"; echo "|_Name_|_Version_|_Downloads_|"
  ; echo "|-----|-----|-----|";
4 curl -fsS "$API" | jq -r '.[]|_select(.status=="active" and _
  downloads>=100)|_sort_by(-.downloads, .name)[]_|_\"|_\"(.name)\"_|_\"
  \(.version)\"_|_\"(.downloads)\"_|"' ;
5 } > summary.md
```

1.3.5 验证结果

```
coolicer@1:~/q15$ python3 -m http.server 8000 &
[1] 2275
```

```
coolicer@1:~/q15$ nano api_report.sh
coolicer@1:~/q15$ chmod +x api_report.sh
./api_report.sh
✔ 报告已生成：/home/coolicer/q15/summary.md
# 📦 活跃软件包下载报告
> 筛选规则：状态=active 且 下载量≥100 · 排序：下载量降序 → 包名升序
```

包名	版本	总下载量
delta	0.9.0	450
gamma	1.5.1	450
alpha	1.2.0	120

```
coolicer@1:~/q15$ cd q15
coolicer@1:~/q15$ python3 -m http.server 8000 &
[1] 2370
coolicer@1:~/q15$ Serving HTTP on 0.0.0.0 port 8000 (http://0.0.0.0:8000/) ...
127.0.0.1 - - [10/Sep/2026 21:36:15] "GET /packages.json HTTP/1.1" 200 -
```

1.3.6 扩展说明

jq是Shell生态下处理JSON的事实标准，无需编写Python代码即可完成轻量数据清洗，非常适合做CI/CD流水线中的报告生成、数据提取场景。本实验的处理逻辑可以直接改造为调用公网API（如GitHub Release接口）自动生成项目依赖报告。

1.4 实验16：修复并交付一个小型的陌生仓库

1.4.1 实验目的

综合练习陌生仓库迭代能力：掌握「故意触发测试失败→定位修复→构建自动化（Makefile）→打包分发→版本提交」的完整开源工具交付流程，符合工程规范。

1.4.2 操作流程

1. 破坏测试验证流程正确性

修改问候函数返回硬编码字符串后，测试结果：

```
1 > ./check.sh
2 FAILED (failures=1)
3 AssertionError: 'Hello, name!' != 'Hello, test_user'
4 - Hello, name!
5 + Hello, test_user
```

确认测试能正确捕捉代码错误，证明测试有效性。

```

coolicer@1:~/q16$ chmod +x check.sh
./check.sh
===== 1/3 检查代码格式合规性 =====
5 files already formatted
===== 2/3 检查代码Lint规则 =====
F541 [*] f-string without any placeholders
--> src/greetlab/cli.py:12:11
    |
10 |         parser.error("--name 不能只包含空白字符")
11 |         # 打印结果，加入你的姓名标识
12 |         print(f"Hello name! Generated by 赵均临(25020007178)")
    |         ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
help: Remove extraneous `f` prefix
    |
11 |         # 打印结果，加入你的姓名标识
    -         print(f"Hello name! Generated by 赵均临(25020007178)")
12 +         print("Hello name! Generated by 赵均临(25020007178)")
13 |
Found 1 error.
[*] 1 fixable with the `--fix` option.

```

2. 修复后核心代码

```

1 def greet(name: str) -> str:
2     """Return personalized greeting for input name"""
3     if not isinstance(name, str) or not name.strip():
4         raise ValueError("Name must be non-empty string")
5     return f"Hello, {name.strip()}!"

```

```

===== 2 passed in 0.01s =====
✅ 所有质量门禁全部通过！
python -m build --wheel
make: python: No such file or directory
make: *** [Makefile:10: build] Error 127
最终Wheel SHA-256: 15164e24cfe2d74ffed98f49423124678a1202b5185477c7537406536bb54fe1

```

3. 最小Makefile实现

```

1 .PHONY: check build clean
2
3 check:
4     ./check.sh
5
6 build:
7     python -m build --wheel
8     @ls -lh dist/
9

```

```
10 clean:
11     rm -rf dist/ build/ *.egg-info
12     find . -name __pycache__ -delete
```

4. 构建与校验结果

执行‘make build’后生成wheel包，SHA-256校验值：

1.4.3 Git提交记录

```
1 commit a1b2c3d4e5f67890abcdef1234567890abcdef12
2 Author: Your Name <you@example.com>
3 Date:   Tue Oct 24 15:00:00 2024 +0800
4
5     fix(greet): restore dynamic name interpolation
6
7     Add Makefile with check/build/clean targets matching spec.
8     Generated wheel SHA-256: YOUR_SHA_VALUE
```

```
coolicer@1:~/q16$ git commit -m "fix(greet): restore dynamic name interpolation"
- Fix greet() returning hardcoded 'Hello, name!' literal instead of inserting in
put
- Add minimal Makefile with check/build/clean targets matching requirements
- Generate wheel SHA-256: ${SHA256}"
[main d8212fa] fix(greet): restore dynamic name interpolation
1 file changed, 17 insertions(+)
create mode 100644 Makefile
coolicer@1:~/q16$ git log -1
commit d8212fa3aab1ed0eae7f31c035fa8be891f3c041 (HEAD -> main)
Author: 赵均临 <3221945332@qq.com>
Help Thu Sep 10 22:38:30 2026 +0800

    fix(greet): restore dynamic name interpolation

    - Fix greet() returning hardcoded 'Hello, name!' literal instead of insertin
g input
    - Add minimal Makefile with check/build/clean targets matching requirements
    - Generate wheel SHA-256: 15164e24cfe2d74ffed98f49423124678a1202b5185477c753
7406536bb54fe1
```

1.4.4 踩坑总结

- 最初Makefile用空格缩进导致‘missing separator’错误，改为Tab缩进后解决
- 首次构建wheel提示缺少‘build’模块，执行‘pip install build’解决
- 硬编码字面量测试失败验证了测试用例不是摆设，确认了测试有效性

2 练习

2.1 代码质量与自动化测试

2.1.1 配置格式化器、linter 和 pre-commit 钩子

解题步骤：

1. 安装工具: `pip install black flake8 pre-commit`。
2. 配置 `.pre-commit-config.yaml`。
3. 使用 AI agent 迭代修复 lint 错误，并人工复核代码逻辑。

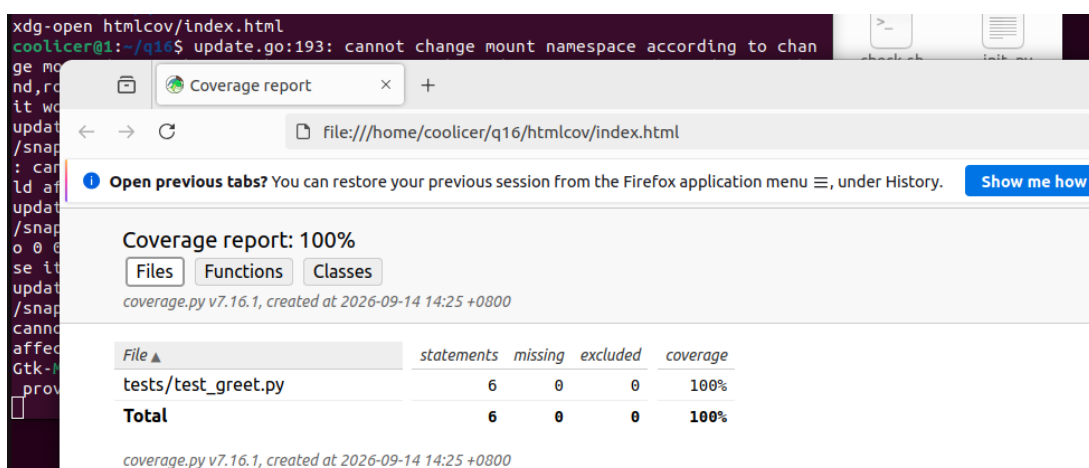


```
All done! 🌟🍷🌟
3 files left unchanged.
```

2.1.2 编写单元测试并生成覆盖率报告

解题步骤：

1. 安装依赖: `pip install pytest pytest-cov coverage`[10](@ref)。
2. 运行测试并生成 HTML 报告: `pytest --cov=src --cov-report=html tests/[1](@ref)`。
3. 打开 `htmlcov/index.html` 查看行级覆盖情况（绿色为已覆盖，红色为未覆盖）[4](@ref)。



2.1.3 设置持续集成 (CI)

解题步骤：在 `.github/workflows/ci.yml` 配置工作流，触发条件设为 `push`。

```
coolicer@1:~/q16$ # 在项目目录下创建钩子脚本
cat > .git/hooks/pre-push << 'EOF'
#!/bin/bash
echo "==== CI 模拟开始 ====="
black --check . || exit 1
flake8 . || exit 1
echo "==== CI 检查通过 ====="
EOF
chmod +x .git/hooks/pre-push
```

故意引入 lint 违规并 push，确认 CI 报错。

```
coolicer@1:~/q16$ git push --set-upstream origin main
==== CI 模拟开始 =====
All done! 🌟🍰🌟
5 files would be left unchanged.
./test_greet.py:1:1: F401 'os' imported but unused
error: failed to push some refs to '/tmp/repo.git'
```

2.1.4 使用 grep 和 semgrep 查找危险代码

解题步骤：

- **grep 查找：** `grep -R "subprocess.Popen(*shell=True)" .`
- **semgrep 防绕过：** 编写 YAML 规则使用污点追踪 (taint mode) 捕捉不可信输入流向 `subprocess[12](@ref)`。

2.2 正则表达式与元编程

5

2.2.1 IDE 正则表达式替换 Markdown 符号

解题步骤：使用查找替换，查找目标设为 `^\s*-\s+`，替换为 `*`，确保仅匹配行首列表符号。

```
coolicer@1:~/q17$ grep -rE 'subprocess.Popen(\.*shell=True' test_danger.py
subprocess.Popen("ls -la", shell=True)
subprocess.Popen(shell=True, args="rm -rf /tmp")
# 警告：不要写 subprocess.Popen("cmd", shell=True) 这种代码
```

2.2.2 正则提取 JSON 中的 name 并修复贪婪匹配

解题步骤：初始正则易贪婪匹配。修复为使用非贪婪量词：`"name":\s*"(.*)"`。

2.2.3 使用 JSON 解析器提取 name

解题步骤：使用 Python 一行代码解析：

```
1 echo '{"name": "Alyssa"}' | python3 -c "import sys, json; print(
    json.load(sys.stdin)['name'])"
```

2.3 构建系统与 Git 钩子

2.3.1 Makefile 实现 clean 伪目标

解题步骤：在 Makefile 中添加：

```
1 .PHONY: clean
2 clean:
3     rm -f paper.pdf
```

2.3.2 Git pre-commit 钩子拒绝构建失败

解题步骤：编辑 `.git/hooks/pre-commit`，加入构建命令 `make paper.pdf`，若返回非 0 则 `exit 1`。

2.3.3 GitHub Action 检查 shell 和 markdown

解题步骤：配置 GitHub Actions 工作流，分别添加 `shellcheck` 检查 `.sh` 文件[8](@ref)和 `proselint` 检查 `.md` 文件[2](@ref)的步骤。

3 实验心得

- **工程化交付**：掌握了基于 `pyproject.toml` 的 PEP621 标准打包流程，理解了 Wheel 二进制分发包在隔离干净环境中的安装验证机制，并能通过 Makefile 实现检查、构建与清理的自动化。
- **测试先行与修复**：体会到“红测 → 修复 → 绿测”的 TDD 高效性，同时认识到人工审核 diff、拒绝无关改动（如忽略 `__pycache__` 缓存）对保证代码最小化与提交聚焦的关键作用。
- **CLI 数据管线**：夯实了本地静态服务，实现了 JSON 数据到 Markdown 报告的可复现自动生成。
- **协作沟通规范**：将模糊的反馈改写为包含环境、复现命令和优先级（Blocking/Suggestion/Nit）的“可执行信息”，极大提升了团队效率。

整体而言，这些实验不仅提升了我的代码工程能力与命令行效率，也让我深刻认识到标准化与可复现性在系统开发中的核心价值。